

DigitalTwin.ai — Project Guide

Accenture Innovation Challenge 2026 · Problem Statement 4 · Team HipHipHooray — Sagar Sahu, Priyansh Goyal. Everything needed to pick this project up on another machine.

1 · WHAT THE PROJECT IS

A live digital twin of a vehicle assembly line that shows where bottlenecks are forming and predicts defects before they happen. We cleared Round 1 and are now building the Round 2 prototype.

The thesis in one line: a plant is instrumented for *utilisation*, but utilisation does not tell you what is costing you cars. We rank stations by what fixing them is worth, in cars — not by how busy they look.

PS4 states three problems, and our solution answers them one-for-one:

PROBLEM (FROM THE BRIEF)	OUR ANSWER	KEY MEASURED RESULT
A bottleneck ripples downstream	Read the blocked / starved signature; rank by processing time ÷ uptime	46% / 58% top-1/top-2 vs 35% / 48% for utilisation
An early defect goes uncaught for dozens of vehicles	Watch every process channel for drift, then classify <i>why</i>	421–1,322 vehicles of warning before scrap
Stations with little or no sensor data	Bracket between neighbours, then correct the bias bracketing introduces	88.8 s raw → 58.0 s true (53% overstatement removed)

2 · GETTING SET UP ON A NEW MACHINE

```
git clone https://github.com/sagar2907/HipHipHooray_DigitalTwin.ai.git
cd HipHipHooray_DigitalTwin.ai
pip install numpy pandas matplotlib pytest
```

Or use `6_Code/digitaltwin_code_snapshot.zip` in this folder — identical contents, taken from the committed tree.

THE DATASETS ARE NOT IN THE REPO

They are ~765 MB of generated data and are deliberately gitignored. The *generators* are what is versioned. Regenerate with the builders in `scripts/` — they take fixed seeds, so the data reproduces bit-for-bit. Budget a couple of hours for the full v5 truth build.

What lives where

PATH	CONTENTS
src/twin/	11 modules. <code>plant.py</code> is the simulator, <code>detect.py</code> the bottleneck detector, <code>tools.py</code> the tool telemetry model, <code>dark.py</code> the no-sensor estimator, <code>forming.py</code> the buffer countdown.
scripts/	18 scripts: dataset builders, the ground-truth builder, and five evaluation harnesses.
tests/	11 tests covering CRN pairing, leakage and conservation. One currently fails — see §5.
results/	Every number quoted in the deck, with the CSV that produced it.
reports/	Earlier PDF write-ups. Superseded by the two design documents in <code>4_Design_Documents/</code> .

THE REPO README IS OUT OF DATE

It documents the *original* single-dataset experiment (6 nutrunners, CUSUM at 423 vehicles). The current work is v3 (process telemetry) and v5 (flow). Read the design document in `4_Design_Documents/` as the source of truth, not the README.

Pipeline 1 — Bottleneck formation

- Every station carries a **working / blocked / starved** state — measured where a PLC tag exists, estimated where it does not.
- The constraint is the station that runs flat out while the one *behind* it blocks and the one *ahead* starves. That three-station signature is what locates it — not raw busyness, which is the utilisation trap.
- Rank by **processing time ÷ uptime** (effective cycle time), then pull only that one station's full history. Cheap checks everywhere, expensive checks only on suspicion.
- Buffer fill-rates flag the *next* constraint before it forms — 60% of warnings are followed by a real block.

Pipeline 2 — Defect prediction

- Track always-on signals per station — torque, current, cycle time — using running statistics (CUSUM / EWMA) that cost two numbers of state per channel.
- Combine into one health score; stay silent unless it drifts from *that station's own* healthy baseline.
- **Classify why before acting.** A worn tool needs servicing; a tool whose sensor merely drifted needs only recalibrating. Get it wrong and you either pass 11,631 bad joints or scrap 700–960 good ones.
- Before raising a work order, check whether every other tool on the same material lot alarmed too — 21 of 23 did on a bad lot, and 0 on other lots.

Pipeline 3 — Stations with no sensors

- Bracket a silent station between the two timestamps we do have, minus an *estimated transit-time distribution* — never a fixed number.
- Cross-check the neighbours: an empty upstream buffer means it was waiting, not working. Bracketing alone overstates real work by 53%.
- Several dark stations in a row? Locate the culprit from arrival delays on each side, the way a seismograph places an earthquake.
- Every estimate ships a confidence score. A fully opaque section gets a *costed sensor recommendation* rather than a guess.

The loop that ties them together

Line → event spine → twin (replay + paired counterfactual) → ranked call in cars gained → supervisor confirms → the twin re-reads the line. **It advises; it never writes to line control.** That boundary is deliberate: writing to safety-certified line control is a regulated change no plant grants a prototype.

4 · WHAT IS BUILT, DESIGNED, AND FAILED

Built and measured

- Plant simulator: 20 stations, rework loops, shift calendar, andon, changeover, parallel stations
- Tool telemetry: 4 families, 12 fault conditions
- All three detection pipelines, scored against hidden ground truth
- Held-out threshold calibration + a Wiener-process firewall set

Designed, not built

- Decision surface (station record, alert contract, observability map)
- The live loop — everything today is batch
- Twin drift metric and re-calibration trigger
- Genealogy containment; failure-mode classifier
- Stop-or-continue joint decision

Tested and failed — report these, do not hide them

- **Overtake risk:** stated 70–100% confidence, right 5.9% of the time. Linear extrapolation of an early trend. The same error made remaining-life estimates $\sim 3\times$ too long.
- **Dark localisation:** only 5 of 142 blocks contain the constraint, so no threshold can be calibrated. A dataset gap, not a verdict on the method.

5 · KNOWN PROBLEMS TO FIX FIRST

ONE TEST IS FAILING

`test_speedup_never_hurts_much` — speeding a station up loses 5 cars (339→334). Sweeping all 20 stations shows S00–S05 all lose exactly 5 and S06–S11 exactly 2, which is not what physical coupling produces. It points at RNG desynchronisation in the global `speed_scale` path.

IT DOES NOT AFFECT OUR PUBLISHED NUMBERS.

The sensitivity truth is built by `build_truth.py` through the `windowed_perturb=` path, which has its own passing test. `speed_scale` is used only by the superseded v2 builder. Fix or delete it — and correct the `plant.py` docstring, which still claims speed-perturbation "changes no other draw".

- **Noise floor unquantified.** Paired comparisons carry several cars of noise. The healthy-run figures (top gain 7.0, margin 3.8) sit near it; the faulted-run figures (18.0, 12.0) are clear of it. Measure the floor before quoting the healthy-run numbers.

6 · SIX RULES WE EARNED BY CATCHING OUR OWN MISTAKES

- **Never let a detector recompute its own ground truth.** We got this wrong three times. Truth is defined economically — which station's speed-up actually produces more cars.
- **Calibrate on held-out data.** "Zero false alarms" was in-sample; the real rate is about one per five tool-weeks.
- **A number is measured with a named source file, or cited with a named reference, or it does not appear.** This caught an unsourced claim before it reached a slide.
- **Report negative results as prominently as positive ones.**
- **Two statistics with opposite signs are one statistic.** Utilisation and least-idle are the same measure; reporting their agreement was an identity, not a result.
- **Prove the modelling boundary by ablation, don't assert it.** The shift calendar looked cosmetic until it turned out to be the only thing that drains a backlog.

7 · WHAT IS IN THIS FOLDER

FOLDER	CONTENTS
1_Guide	This document.
2_Plan	The Round 2 plan of action.
3_Round1_Submission	The deck (PPTX + PDF) and the 2:53 pitch video.
4_Design_Documents	The full pipeline design (21 pp), the evidence file (6 pp, per-claim provenance), and the Round 2 problem statements.

5_Animation_Sources	Four self-contained HTML animations (open in Chrome), the silent captioned MP4, and the result figures.
6_Code	Snapshot zip of the repo; GitHub link for the live version.
7_Prototype	Round 2 prototype work.

THE SINGLE MOST USEFUL THING TO UNDERSTAND

Everything we have today is

BATCH

— scripts that read a finished dataset and print a score. That is a digital *shadow*, not a twin. Closing the loop so it ingests, detects, ranks and re-reads on a timer is the whole difference, and it is the claim a Round 2 judge will probe hardest.